FATIMA JINNAH WOMEN UNIVERSITY
RAWALPINDI

Cloud DevOps



Lab 06

Set Up Nexus Repository and Configure User Access

Submitted by:	Rabbia Noor
Registration No.:	2023-BCS-070
Course Code:	CS-649
GitHub Username:	Rabbia-Noor
Section:	A
Repository:	github.com/Rabbia-Noor/DevOps (Labs/Lab06)

Task 1: Run Nexus Repository and Complete the Initial Setup

Step 1: Verify the Nexus container is running



docker ps | grep nexus shows the nexus container Up with ports 8006 and 8007 published.

Task 2: Create a Role and User and Verify Access

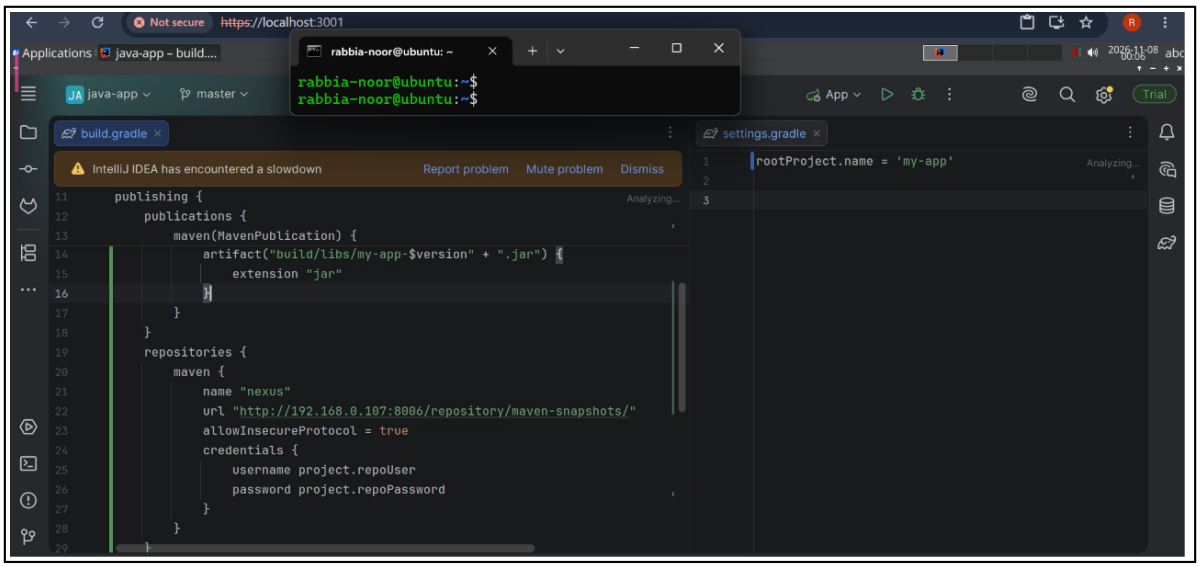
Step 1: Verify the nexus user access



The Status API returns HTTP/1.1 200 OK, so the nexus user is active and authenticates without showing the password.

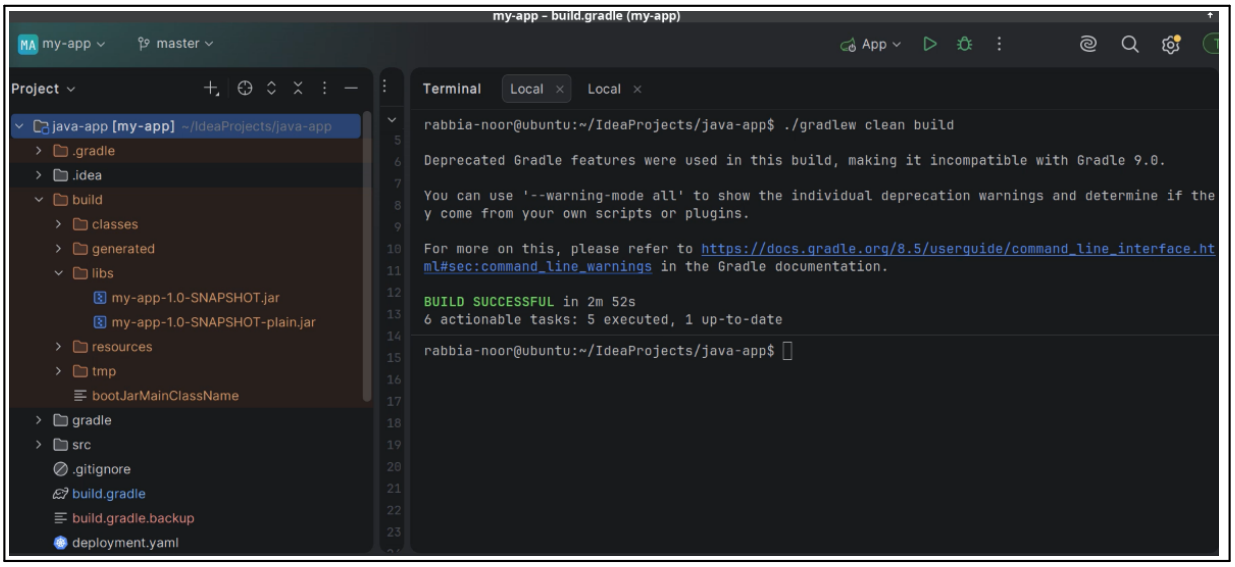
Task 3: Build and Publish the Gradle Application to Nexus

Step 1: Configure build.gradle and settings.gradle



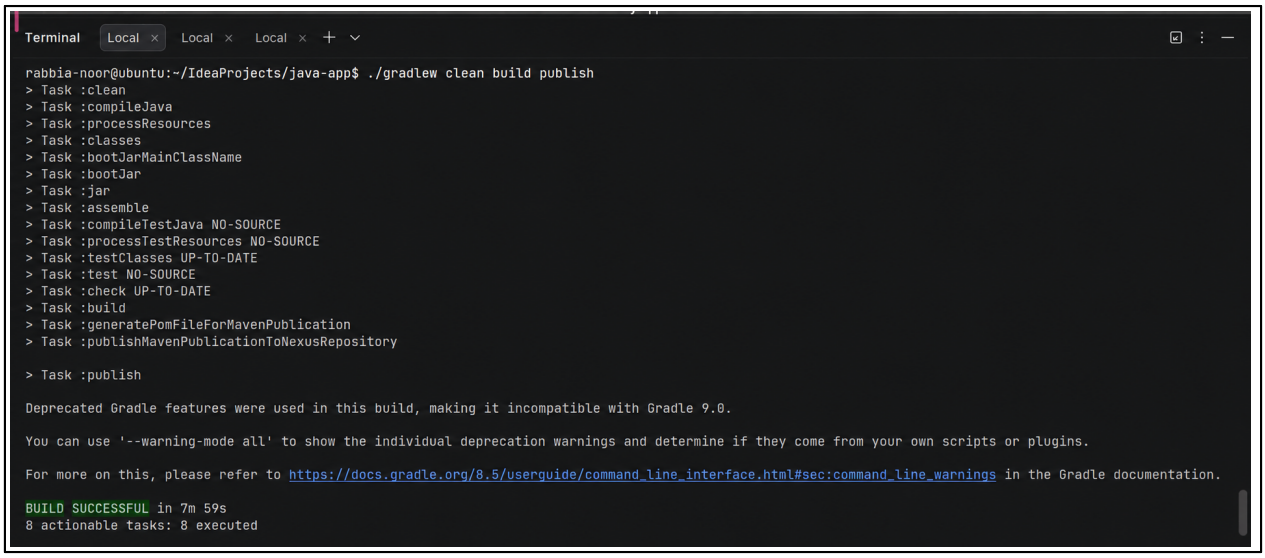
The publishing block points to the Nexus maven-snapshots URL and settings.gradle sets the project name to my-app.

Step 2: Build the JAR



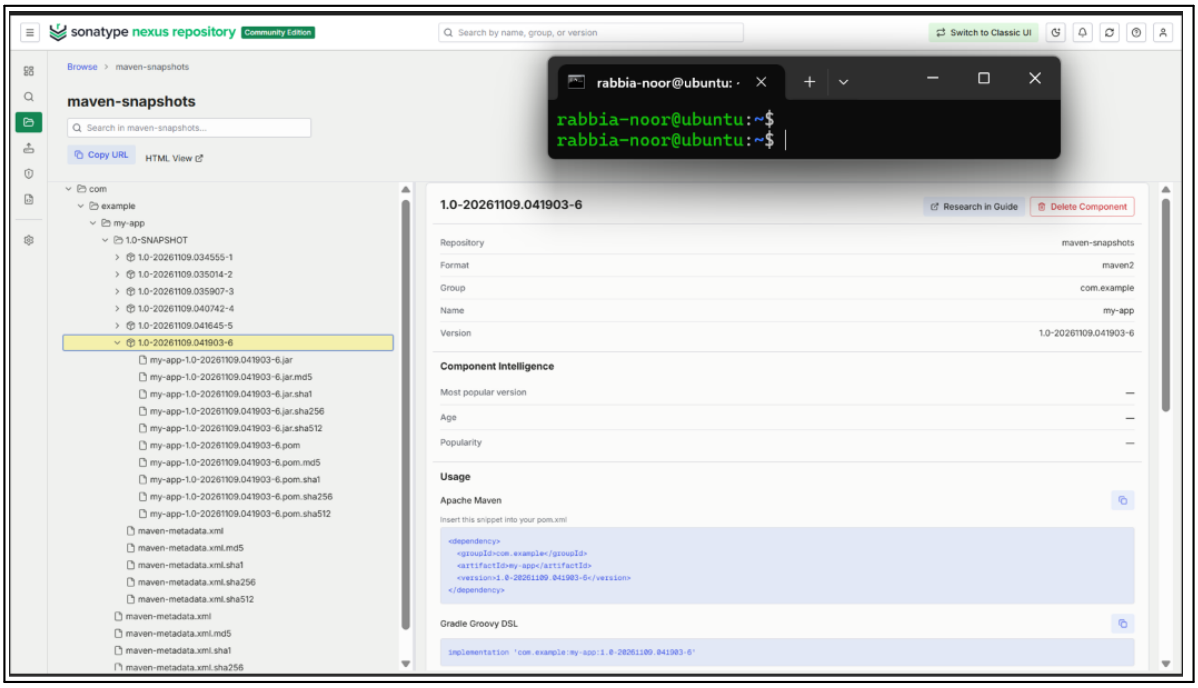
./gradlew clean build ends with BUILD SUCCESSFUL and creates my-app-1.0-SNAPSHOT.jar in build/libs.

Step 3: Publish the JAR to Nexus



./gradlew clean build publish runs the Nexus publish task and ends with BUILD SUCCESSFUL.

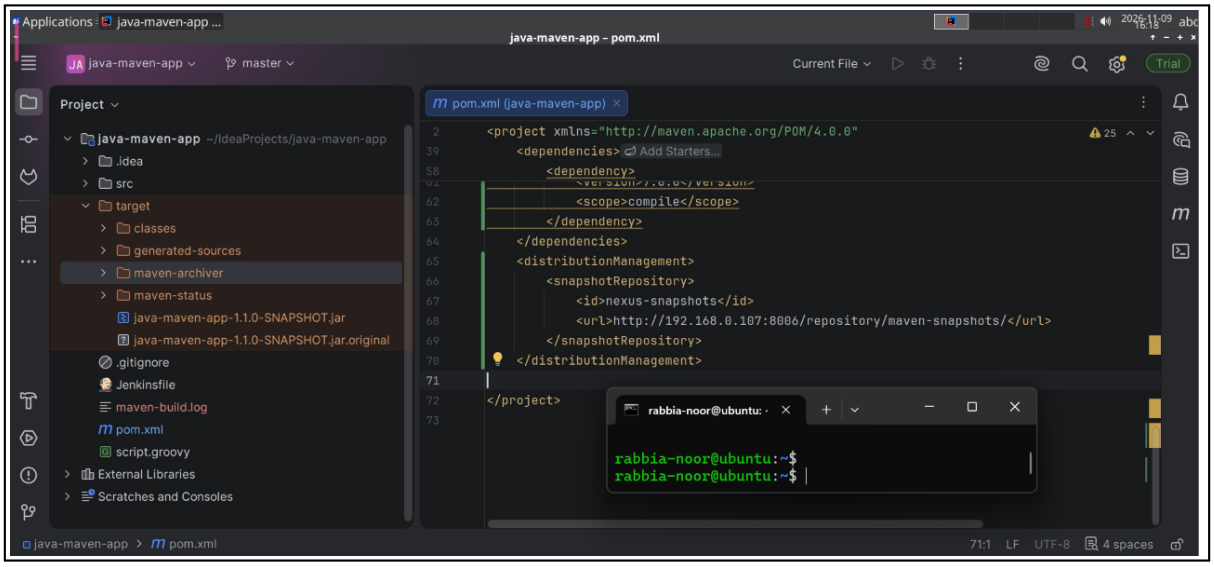
Step 4: Verify the artifact in Nexus



maven-snapshots contains com/example/my-app/1.0-SNAPSHOT with the JAR, POM and metadata files.

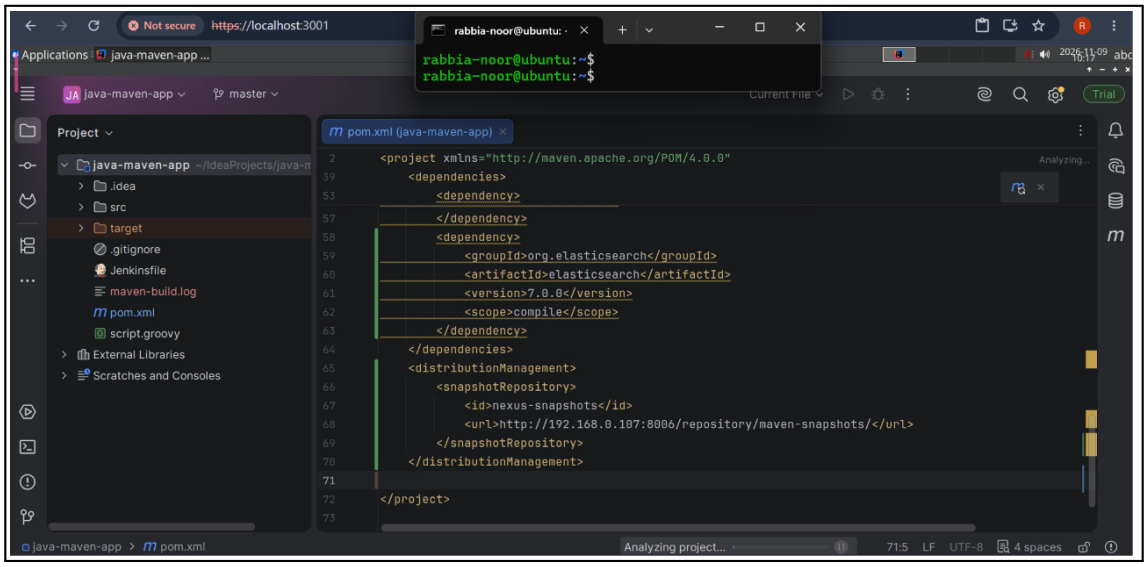
Task 4: Publish the Maven Application to Nexus

Step 1: Open the java-maven-app project



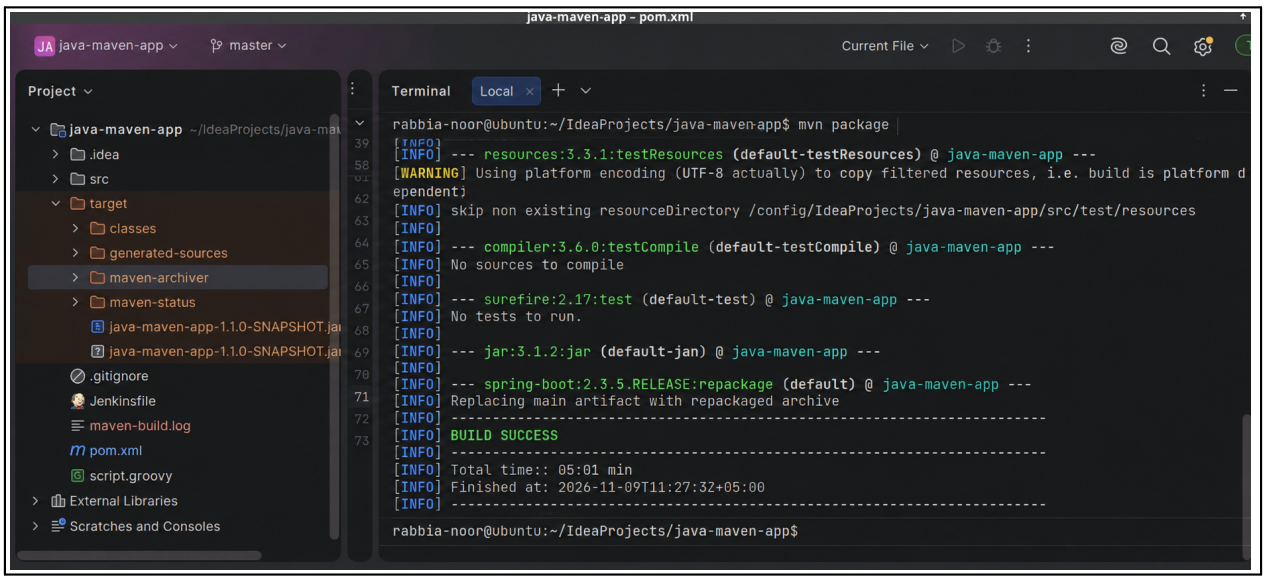
The java-maven-app project is open in IntelliJ IDEA.

Step 2: Add distributionManagement to pom.xml



The snapshotRepository block (id nexus-snapshots) points to the Nexus maven-snapshots URL.

Step 3: Package the application



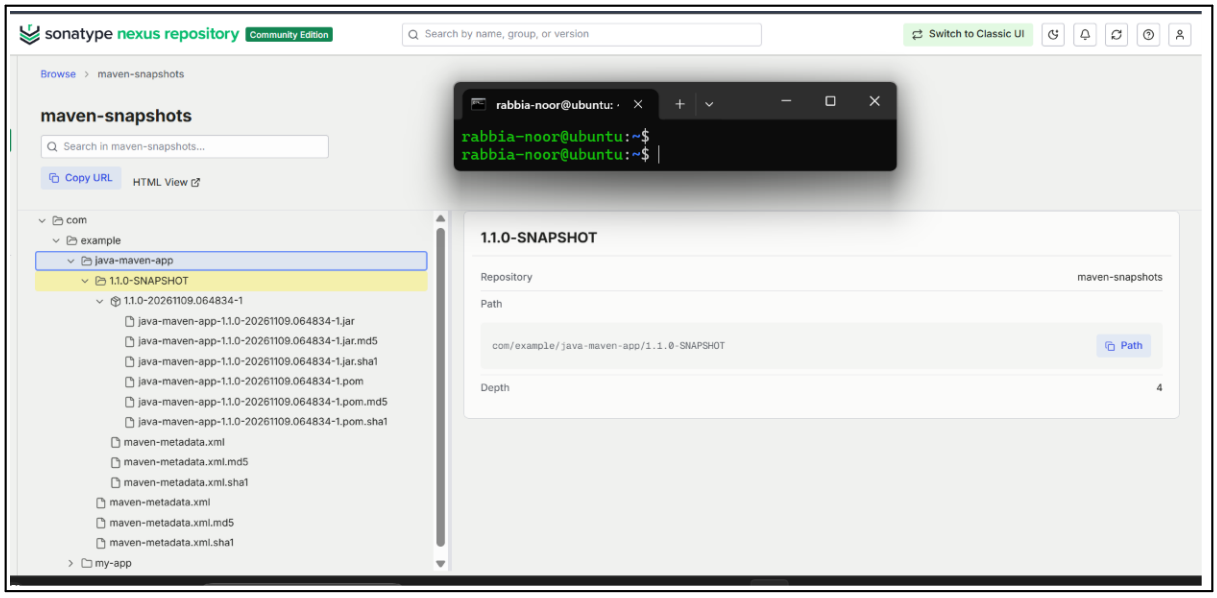
mvn package ends with BUILD SUCCESS and creates java-maven-app-1.1.0-SNAPSHOT.jar.

Step 4: Deploy the snapshot to Nexus

```
rabbia-noor@ubuntu:~/IdeaProjects/java-maven-app$ mvn deploy
[INFO] Scanning for projects...
[INFO]
[INFO] ---------------------< com.example:java-maven-app >---------------------
[INFO] Building java-maven-app 1.1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/3.1.2/maven-install-plugin-3.1.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/3.1.2/maven-install-plugin-3.1.2.pom (8.5 kB at 7
90 B/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/42/maven-plugins-42.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/42/maven-plugins-42.pom (7.7 kB at 23 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/3.1.2/maven-install-plugin-3.1.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/3.1.2/maven-install-plugin-3.1.2.jar (32 kB at 14
 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/3.1.2/maven-deploy-plugin-3.1.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/3.1.2/maven-deploy-plugin-3.1.2.pom (9.6 kB at 15
kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/3.1.2/maven-deploy-plugin-3.1.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/3.1.2/maven-deploy-plugin-3.1.2.jar (40 kB at 51 k
B/s)
[INFO]
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ java-maven-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.6.0:compile (default-compile) @ java-maven-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ java-maven-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /config/IdeaProjects/java-maven-app/src/test/resources
[INFO]
[INFO] --- compiler:3.6.0:testCompile (default-testCompile) @ java-maven-app ---
[INFO] No sources to compile
[INFO]
[INFO] --- surefire:2.17:test (default-test) @ java-maven-app ---
[INFO] No tests to run.
[INFO]
[INFO] --- jar:3.1.2:jar (default-jar) @ java-maven-app ---
[INFO]
[INFO] --- spring-boot:2.3.5.RELEASE:repackage (default) @ java-maven-app ---
[INFO] Replacing main artifact with repackaged archive
[INFO]
[ ... output omitted ... ]
    at java.lang.Thread.run (Thread.java:1583)
Uploaded to nexus-snapshots: http://192.168.0.107:8006/repository/maven-snapshots/com/example/java-maven-app/maven-metadata.xml (285 B at 0 B/s)
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  14:41 min
[INFO] Finished at: 2026-11-09T12:03:16+05:00
[INFO] ------------------------------------------------------------------------
```

mvn deploy uploads the artifact to nexus-snapshots and ends with BUILD SUCCESS.

Step 5: Verify the artifact in Nexus



maven-snapshots contains com/example/java-maven-app/1.1.0-SNAPSHOT with the JAR, POM and metadata files.

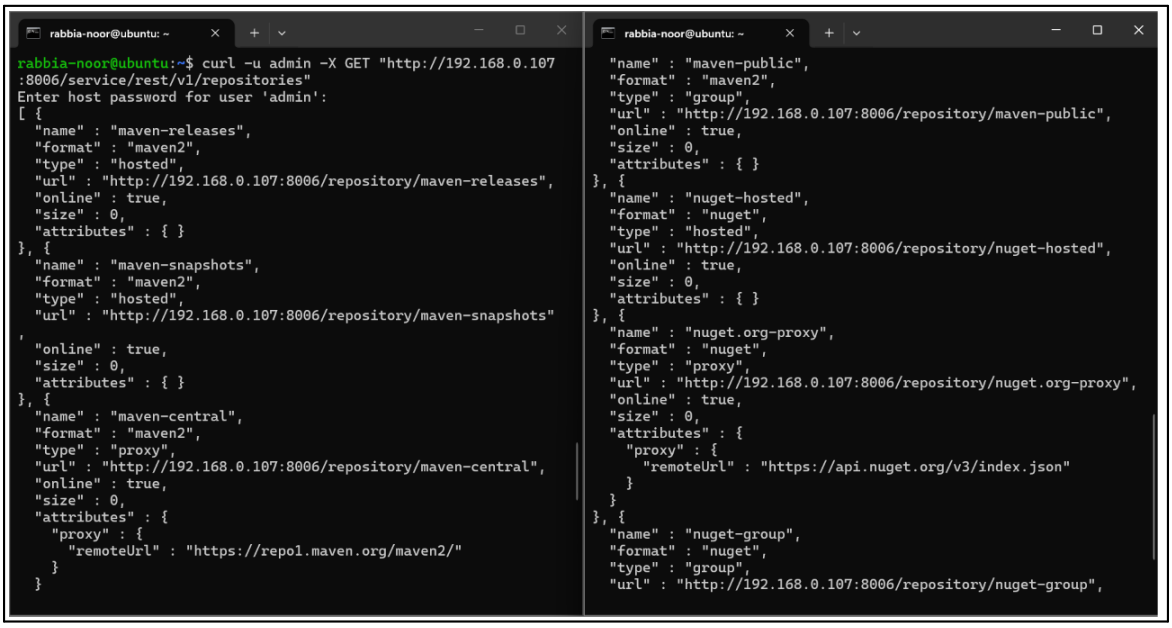
Task 5: Inspect Nexus Repositories and Components Using the REST API

Step 1: List repositories as the nexus user

```
rabbia-noor@ubuntu:~$ curl -u nexus -X GET "http://192.168.0.107:8006/service/rest/v1/repositories"
Enter host password for user 'nexus':
[ {
  "name" : "maven-snapshots",
  "format" : "maven2",
  "type" : "hosted",
  "url" : "http://192.168.0.107:8006/repository/maven-snapshots",
  "online" : true,
  "size" : 0,
  "attributes" : { }
} ]rabbia-noor@ubuntu:~$
```

The restricted nexus user can see only the maven-snapshots repository.

Step 2: List repositories as the admin user



The admin user can see all Nexus repositories.

Step 3: List the components in maven-snapshots



The response lists a component with its id, repository, name and version.

Step 4: Get one component by ID



Requesting the component by its id returns its repository, format, group, name, version and assets.

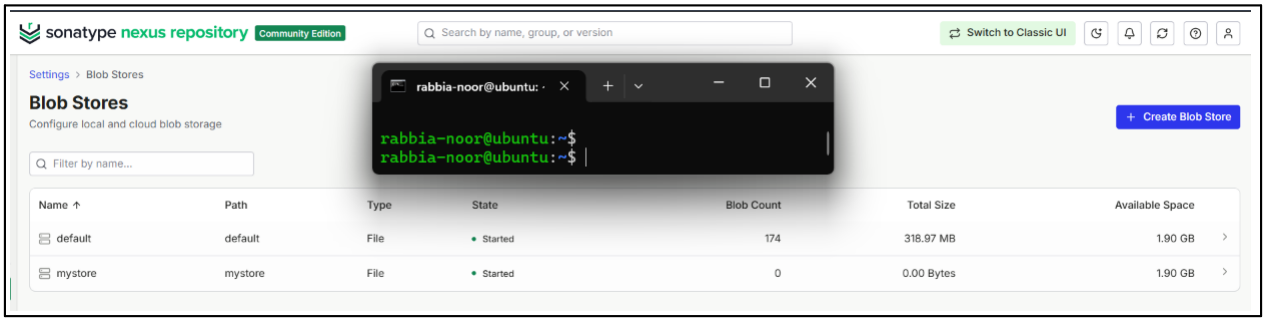
Task 6: Create a Nexus Blob Store and Docker Hosted Repository

Step 1: Inspect the default blob store

```
}rabbia-noor@ubuntu:~$docker exec -it nexus /bin/sh
/opt/sonatype $ pwd
/opt/sonatype
/opt/sonatype $ cd sonatype-work/nexus3
/opt/sonatype/sonatype-work/nexus3 $ cd blobs
/opt/sonatype/sonatype-work/nexus3/blobs $ ls
default
/opt/sonatype/sonatype-work/nexus3/blobs $ ls default/content
2026        directpath  tmp
/opt/sonatype/sonatype-work/nexus3/blobs $
```

Inside the container, the blobs directory contains default and its content folder.

Step 2: Create the mystore blob store



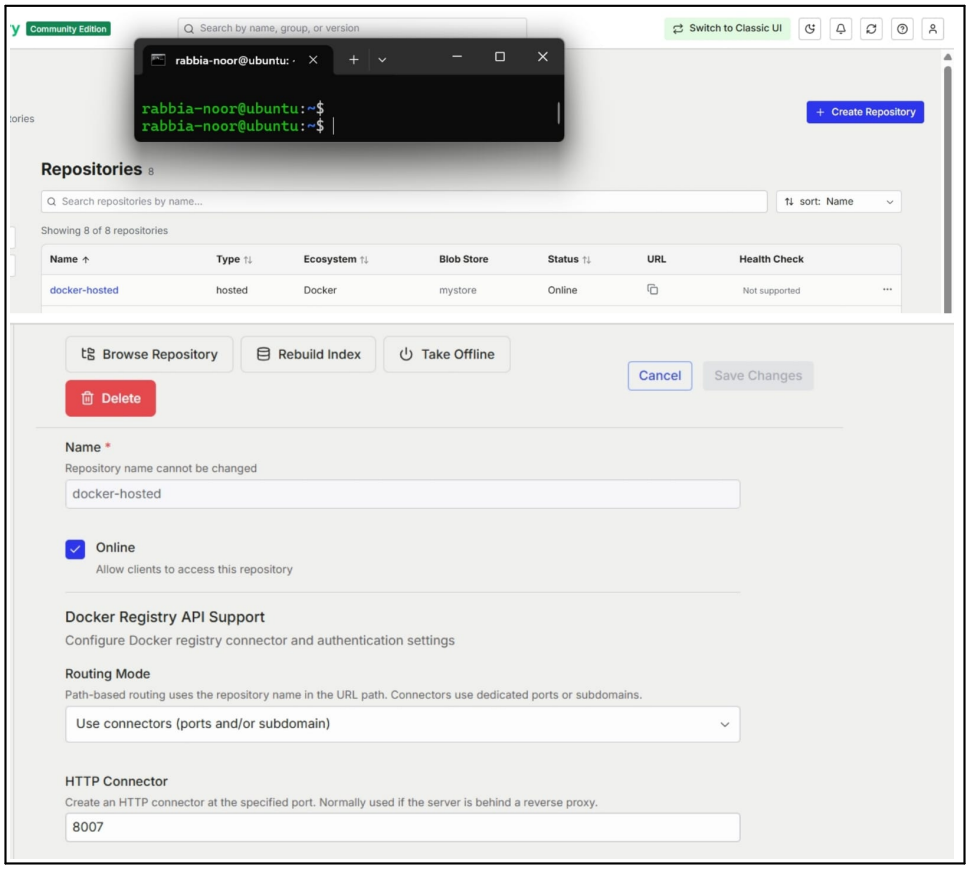
The Blob Stores page lists default and mystore, both File type and Started.

Step 3: Verify the mystore directory

```
/opt/sonatype/sonatype-work/nexus3/blobs $ pwd
/opt/sonatype/sonatype-work/nexus3/blobs
/opt/sonatype/sonatype-work/nexus3/blobs $ ls
default  mystore
/opt/sonatype/sonatype-work/nexus3/blobs $ ls mystore
content              metadata.properties
/opt/sonatype/sonatype-work/nexus3/blobs $ exit
rabbia-noor@ubuntu:~$
```

The blobs directory now contains both default and mystore.

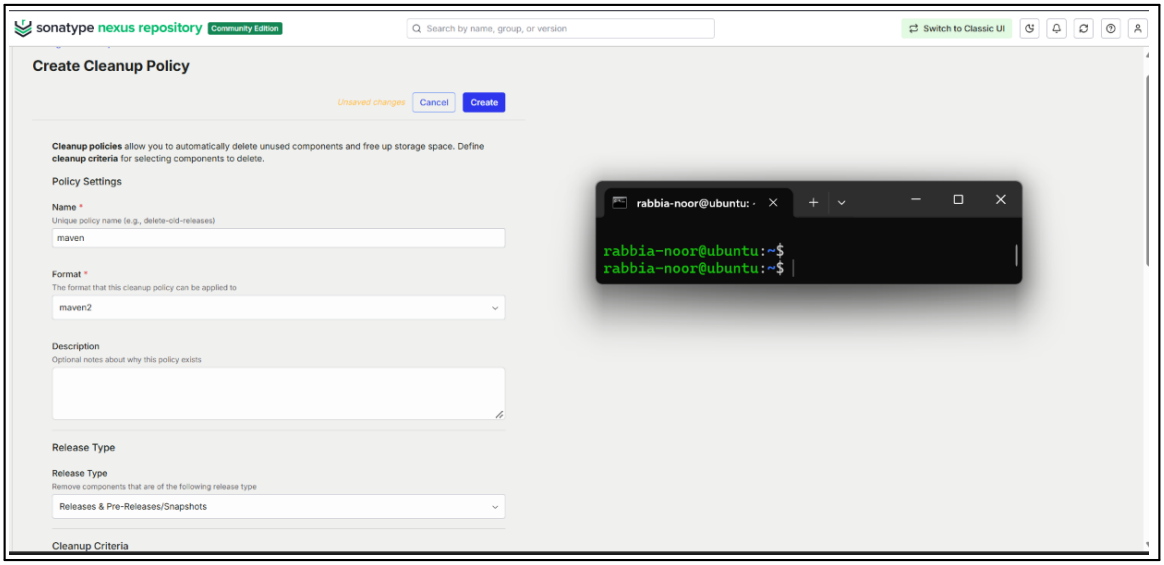
Step 4: Create the docker-hosted repository



docker-hosted is Online, uses the mystore blob store and has HTTP connector 8007.

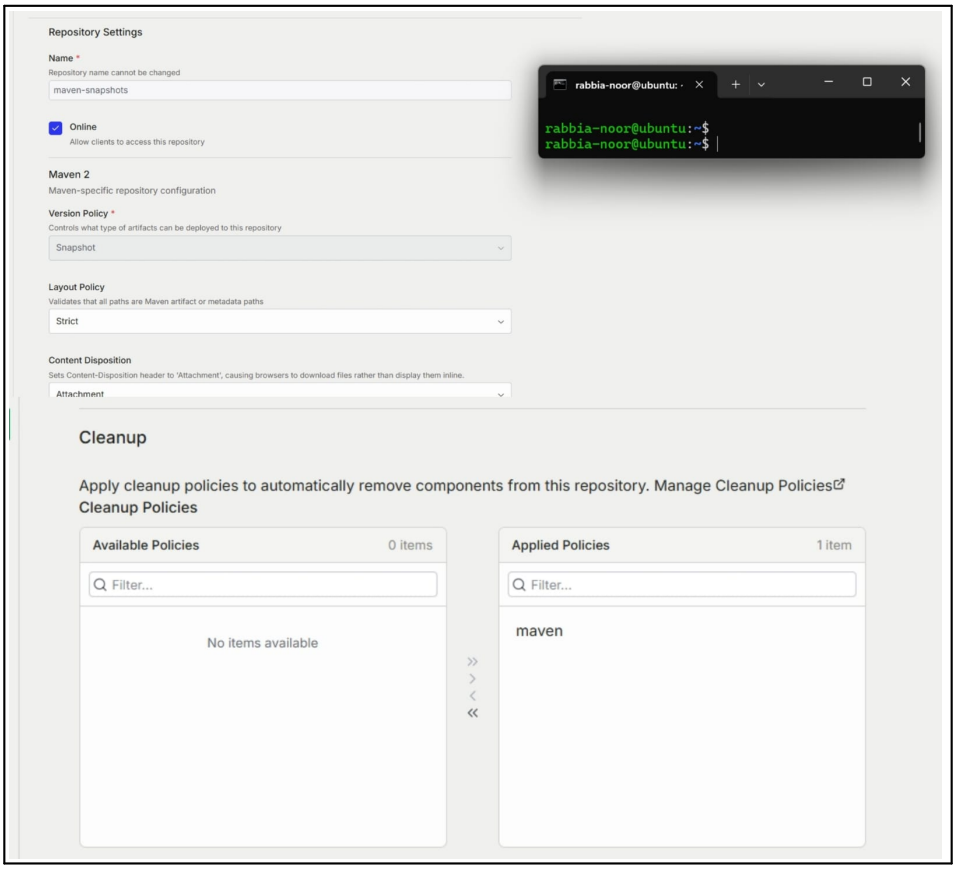
Task 7: Create and Run a Maven Snapshot Cleanup Policy

Step 1: Create the maven cleanup policy



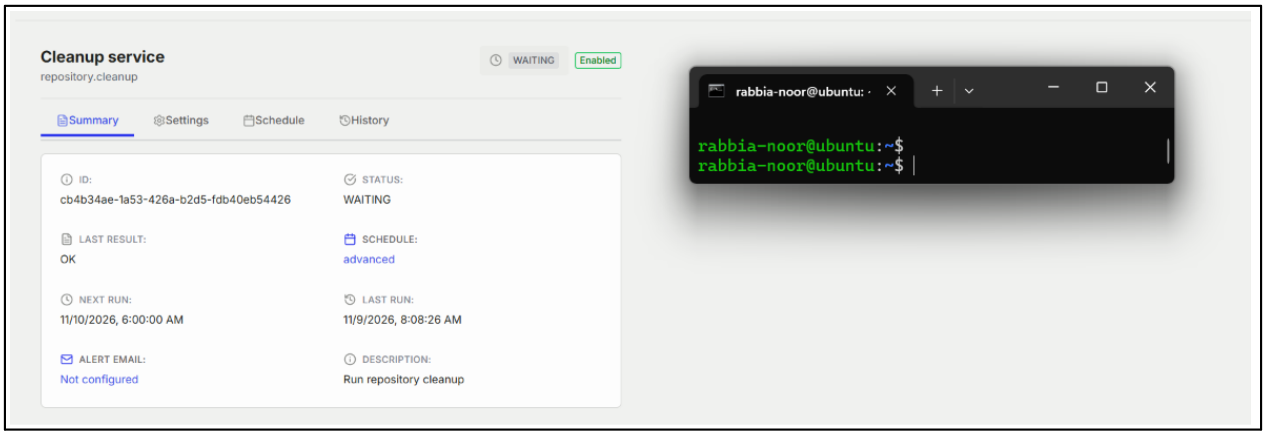
The cleanup policy named maven is created for the maven2 format.

Step 2: Attach the policy to maven-snapshots



The maven policy appears under Applied Policies for maven-snapshots.

Step 3: Verify the cleanup service schedule



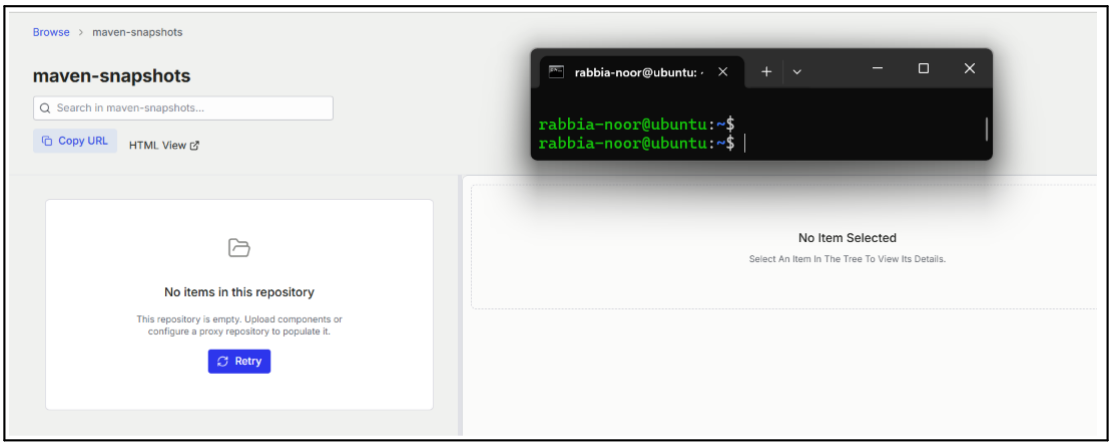
The Cleanup service is Waiting with the next run scheduled at 6:00 AM.

Step 4: Run the cleanup service manually



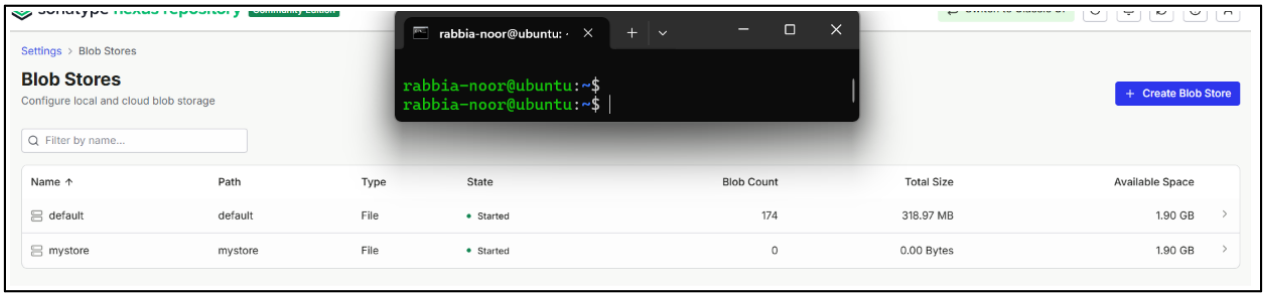
After the manual run the task is Waiting with Last result OK.

Step 5: Verify maven-snapshots is empty



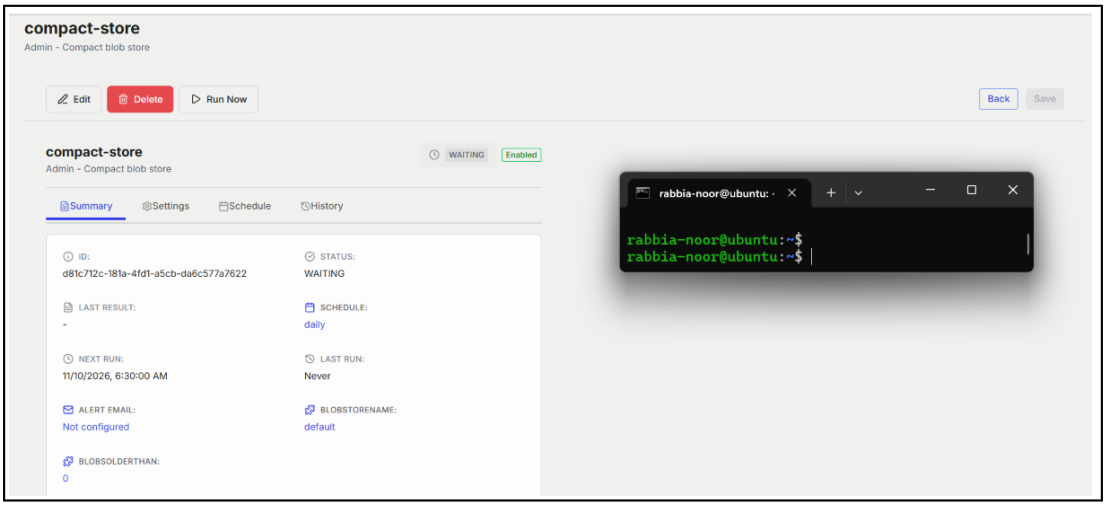
The repository shows no items after the cleanup.

Step 6: Check the blob store before compaction



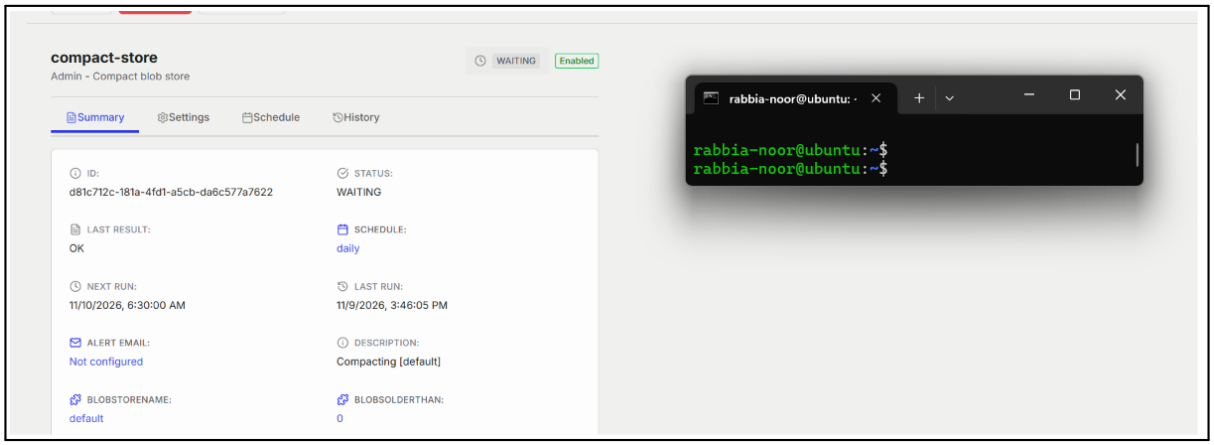
The default blob store still shows 174 blobs and 318.97 MB used.

Step 7: Create the compact-store task



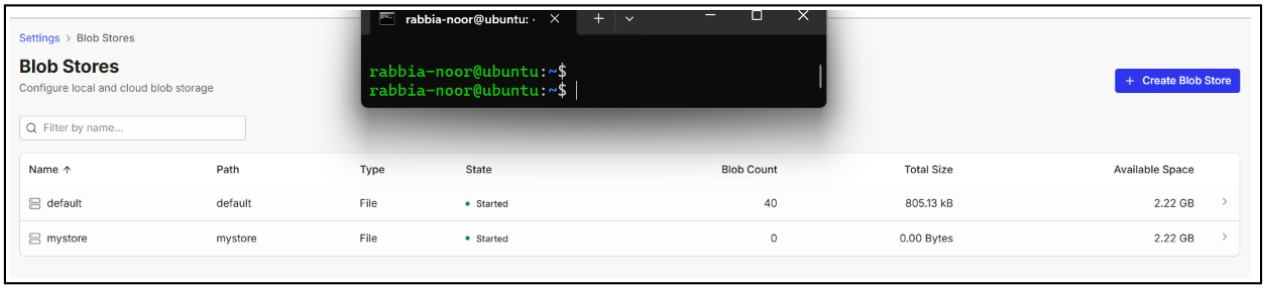
compact-store targets the default blob store with 0 days, scheduled daily at 06:30.

Step 8: Run the compact task manually



After the manual run compact-store is Waiting with Last result OK.

Step 9: Verify space was reclaimed



The default blob store now shows 40 blobs and 805.13 kB used, with more available space.